



T.C
DOKUZ EYLUL UNIVERSITY
FACULTY OF ENGINEERING
ELECTRICAL & ELECTRONICS ENGINEERING DEPARTMENT
EED 1010 ALGORITHMS & PROGRAMMING
BIG CITY

PROJECT FINAL REPORT

by
Ömer Ege Nişli 2024502054

Yiğithan Yıldırım 2024502076

Asal Ghazi Mir Saeid 2022502017

Advisor
Dr. Özlem ÖZTÜRK

09 June 2025
İZMİR

ABSTRACT

This project simulates a city's public transport system using C, featuring 6 bus lines (12 buses) and 16 stops. Key functionalities include:

Dynamic passenger queues with FIFO boarding

Luggage management (0-2 items per passenger)

Real-time terminal UI showing bus positions and statistics

The system demonstrates file I/O (for route data), struct-based memory management, and modular architecture separating data, logic, and UI layers.

TABLE OF CONTENTS

ABSTRACT	2
1. INTRODUCTION	4
1.1 AIM OF THE PROJECT.....	4
1.2 SCOPE.....	4
1.3 FEATURES.....	4
2. WORKING PRINCIPLE	4
2.1 SYSTEM DESIGN.....	4
2.1.1 City Map & Visualization Layer.....	4
2.1.2 Simulation Engine.....	5
2.1.3 Passenger & Bus System.....	6
2.2 DATA STRUCTURES.....	7
2.2.1 Core Structures.....	7
2.2.2 Global Variables.....	7
3. KEY FEATURES	8
4. IMPLEMENTATION	9
4.1 KEY FUNCTIONS.....	9
4.1.1 Data Loading (Initialize City).....	9
4.1.2 Bus Movements (moveBus).....	10
4.2 ALGORITHMS.....	10
4.2.1FIFO Passenger Boarding.....	10
4.2.2Unloading Passengers (unloadPassengers).....	10
4.2.3New Passenger Generation (generatePassenger)....	10
4.3 PROBLEMS FACED.....	11
5. GROUP MEMBERS AND THEIR WORK	11
6. CONCLUSION	11

1. INTRODUCTION

1.1 AIM OF THE PROJECT

To create a real-time bus simulation that accurately models city transportation and provides statistical insights about the system.

1.2 SCOPE

- City grid (21x57) layout
- 6 bus lines with 2 buses each
- 16 bus stops (A–P)
- Passenger generation with random luggage
- Bus operations including luggage loading
- Statistics display (e.g., fullness, wait times)

1.3 FEATURES

- Real-time simulation loop
- Dynamic passenger queue
- Bus line logic and endpoint switching
- Luggage ID tracking
- Statistics tracking panel

2. WORKING PRINCIPLE

2.1 SYSTEM DESIGN

2.1.1 City Map & Visualization Layer

- Text-based visual grid
- Bus stops, buses, and passengers visually marked
- Passenger info and bus capacity displayed on screen

[illegible]

```

> FOCEEPNBMFLLHIE >

New Passengers
-----
Waiting      : 16
Travelling   : 55
Bus fullness: 57.29%
Bus [D2] Passengers:
61: C-P (L:-)
76: D-A (L:-)
79: D-E (L:-)
Bus [D2] Luggage:
| - |
-----

Selected Stop Details
-----
[B: 5 passengers, 5 luggage]
ID From -> To Luggage
48: B -> I 47,48
50: B -> K 49
51: B -> H 50,51
55: B -> P -
68: B -> M -

Keys:
[R] Run continuously
[SPACE] Step by step
[Q] Quit
[W] Select previous stop [E] Select next stop [Current: B]

```

2.1.2 Simulation Engine

- Time controlled with keyboard (run, pause, step)
- Buses move every 2-unit time
- Passengers board if space is available
- Luggage placed/taken takes 1-unit time per piece

```

void moveBus(Bus* bus, BusLine* line, BusStop stops[]) {
    int currentIndex = -1;
    for (int i = 0; i < line->stopCount; i++)
        if (line->stops[i] == bus->currentStop) { currentIndex = i; break; }
    if (currentIndex == -1) return;
    int newIndex = currentIndex + bus->direction;
    if (newIndex < 0 || newIndex >= line->stopCount) {
        bus->direction *= -1;
        newIndex = currentIndex + bus->direction;
    }
    bus->currentStop = line->stops[newIndex];
    unloadPassengers(bus);
    loadPassengers(bus, stops);
}

```

```

void loadPassengers(Bus* bus, BusStop stops[]) {
    int stopIndex = bus->currentStop - 'A';
    BusStop* stop = &stops[stopIndex];
    int maxPassengers = 8 - bus->passengerCount;
    int maxLuggage = 8 - bus->luggageCount;

    while (stop->front <= stop->rear && maxPassengers > 0) {
        Passenger p = stop->queue[stop->front];
        if (p.luggageCount <= maxLuggage) {
            bus->passengers[bus->passengerCount++] = p;
            bus->luggageCount += p.luggageCount;
            maxPassengers--;
            maxLuggage -= p.luggageCount;
            stop->front++;
        }
        else break;
    }
}

```

2.1.3 Passenger & Bus System

- Passengers have source and destination stops
- Each has 0-2 pieces of luggage
- Buses store max 8 passengers and 8 luggage
- Routes are traversed back and forth by each bus

```

Bus fullness: 30.25%
Bus [D2] Passengers:
55: D-P (L:-)
78: K-I (L:80,81)
95: N-L (L:100,101)
110: M-I (L:115)
Bus [D2] Luggage:
| 80 | 81 | 100 | 101 | 115 |
-----

```

```

        else {
            for (int l = 0; l < pa.luggageCount; l++) {
                printf("%d", pa.luggageIDs[l]);
                if (l < pa.luggageCount - 1) printf(",");
            }
            printf("\n");
        }
        printf("\n");
    }

    // Luggage list
    printf("%sBus [%s] Luggage:\n%s", " ", 60, "", SELECTED_BUS_ID, 60, "");
    int yazildi = 0;
    for (int p = 0; p < buses[b].passengerCount; p++) {
        for (int l = 0; l < buses[b].passengers[p].luggageCount; l++) {
            printf("%d | ", buses[b].passengers[p].luggageIDs[l]);
            yazildi = 1;
        }
        if (!yazildi) printf("- |");
        printf("\n%s-----\n", 60, "");
    }

    drawSelectedStopDetails(stops);
    printf("\nKeys:\n[R] Run continuously\n[SPACE] Step by step\n[Q] Quit\n");
    printf("[W] Select previous stop [E] Select next stop [Current: %c]\n", selectedStopID);
}

```

```

void DrawScreen(CityMap* city, BusStop stops[], Bus buses[], PassengerQueue* pq) {
    system("CLEAR");
    drawCityWithBuses(city, stops, buses);
    //drawSelectedStopSummary(stops);
    printf("12345678901234567890123456789012345678901234567 Time: %d (%s)\n",
        currentTime, running ? "running" : "paused");

    // New Passengers
    printf("%sNew Passengers\n%s-----\n", " ", 60, "", 60, "");
    for (int j = 0; j < pq->count; j++) printf("%c", pq->queue[j].startStop);
    printf("\n%s-----\n", " ", 60, "");

    // General statistics
    printf("%sWaiting : %d\n", 60, "", calculateWaitingPassengers(stops));
    printf("%sTravelling : %d\n", 60, "", calculateTravellingPassengers(buses));
    printf("%sBus fullness: %.2f%%\n", 60, "", calculateBusOccupancy(buses));

    // Specific bus display
    printf("%sBus [%s] Passengers:\n", 60, "", selectedBusID);
    for (int b = 0; b < MAX_BUSES; b++) {
        char temp[4];
        sprintf(temp, "%c%d", buses[b].lineName, buses[b].busID);
        if (strcmp(temp, selectedBusID) == 0) {
            for (int p = 0; p < buses[b].passengerCount; p++) {
                Passenger pa = buses[b].passengers[p];
                printf("%s%d: %c-%c (L: ", 60, "", pa.id, pa.startStop, pa.endStop);
                if (pa.luggageCount == 0) printf("-)");
            }
        }
    }
}

```

```

float calculateBusOccupancy(Bus buses[]) {
    int totalPassengers = 0;
    for (int i = 0; i < MAX_BUSES; i++)
        totalPassengers += buses[i].passengerCount;
    return (totalPassengers / (float)(MAX_BUSES * 8)) * 100.0f;
}

```

2.2 DATA STRUCTURES

2.2.1 Core Structs

```
typedef struct {
    char lineName;
    int busID;
    char currentStop;
    int passengerCount;
    Passenger passengers[8];
    int row, col;
    int targetIndex;

    int luggageCount;
    int direction;
} Bus;
```

```
typedef struct {
    int id;
    char startStop;
    char endStop;
    int luggageCount;
    int luggageIDs[2];
} Passenger;
```

2.2.2 Global Variables

```
typedef struct {
    Passenger queue[MAX_QUEUE];
    int count;
} PassengerQueue;
```

```
BusStop stops[STOP_COUNT];
```

```
Bus buses[MAX_BUSES];
```

3. KEY FEATURES

Modular Design:

```
drawScreen(&city, stops, buses, &pq);
char input;
while (1) {
    if (running) {
        simulateStep(&city, stops, buses, &pq);
        Sleep(1300);
    }
    void generatePassenger(BusStop stops[], PassengerQueue* pq) {
        Passenger p;
        p.id = globalPassengerID++;
        p.startStop = getRandomStop();
        do { p.endStop = getRandomStop(); } while (p.endStop == p.startStop);
        p.luggageCount = rand() % 3;
        for (int i = 0; i < p.luggageCount; i++)
            p.luggageIDs[i] = globalLuggageID++;

        int index = p.startStop - 'A';
        if (stops[index].rear < MAX_PASSENGERS - 1) {
            stops[index].rear++;
            stops[index].queue[stops[index].rear] = p;
            if (pq->count < MAX_QUEUE)
                pq->queue[pq->count++] = p;
        }
    }
}

int main() {
    srand(time(NULL));
    CityMap city;
    BusStop stops[STOP_COUNT];
    Bus buses[MAX_BUSES];
    PassengerQueue pq = { .count = 0 };

    int i = 0; i < STOP_COUNT; i++) {
        stops[i].name = 'A' + i;
        stops[i].front = 0;
        stops[i].rear = -1;
    }

    typedef struct {
        char name;
        Passenger queue[MAX_PASSENGERS];
        int front;
        int rear;
    } BusStop;
```

```

void LoadPassengers(Bus* bus, BusStop stops[]) {
    int stopIndex = bus->currentStop - 'A';
    BusStop* stop = &stops[stopIndex];
    int maxPassengers = 8 - bus->passengerCount;
    int maxLuggage = 8 - bus->luggageCount;

    while (stop->front <= stop->rear && maxPassengers > 0) {
        Passenger p = stop->queue[stop->front];
        if (p.luggageCount <= maxLuggage) {
            bus->passengers[bus->passengerCount++] = p;
            bus->luggageCount += p.luggageCount;
            maxPassengers--;
            maxLuggage -= p.luggageCount;
            stop->front++;
        }
        else break;
    }
}

```

Interactive UI:

```

Keys:
[R] Run continuously
[SPACE] Step by step
[Q] Quit
[W] Select previous stop [E] Select next stop [Current: B]
|

```

R: Run/pause simulation

Space: Step-through mode

W/E: Select stops

Statistical Output:

```

> FOCEEPNBMFLLHIE >

New Passengers
-----
Waiting      : 16
Travelling   : 55
Bus fullness: 57.29%
Bus [D2] Passengers:
61: C-P (L:-)
76: D-A (L:-)
79: D-E (L:-)
Bus [D2] Luggage:
| - |
-----

Selected Stop Details
-----
[B: 5 passengers, 5 luggage]
ID  From -> To  Luggage
48:  B -> I    47,48
50:  B -> K    49
51:  B -> H    50,51
55:  B -> P    -
68:  B -> M    -

```


4. IMPLEMENTATION

4.1 KEY FUNCTIONS

4.1.1 Data Loading (Initialize City)

```
c
void initializeCity(CityMap* map) {
    // Initialize 21x57 grid
    for (int i = 0; i < MAP_ROWS; i++) {
        for (int j = 0; j < MAP_COLS; j++) {
            if (i % 6 == 2 || j % 10 == 5) map->grid[i][j] = ' '; // Roads
            else map->grid[i][j] = '#'; // Buildings
        }
    }
    // Place bus stops (A-P)
    map->grid[2][5] = 'A'; map->grid[8][15] = 'E'; // ...etc
}
```

Description:

This function initializes the city map as a 2D grid of size 21×57. Roads are represented with spaces (' ') and buildings with '#'. Bus stops labeled A–P are strategically placed across the grid, forming the transportation network. The map serves as the visualization base used by `drawCityWithBuses()`.

4.1.2 Bus Movements (moveBus)

```
c
void moveBus(Bus* bus, BusLine* line) {
    int nextStopIdx = getCurrentStopIndex(bus, line) + bus->direction;
    if (nextStopIdx < 0 || nextStopIdx >= line->stopCount) {
        bus->direction *= -1; // Reverse at endpoints
        nextStopIdx += 2 * bus->direction;
    }
    bus->currentStop = line->stops[nextStopIdx];
}
```

Description:

Each bus moves step-by-step along its line. It reverses direction upon reaching either

end of the route. After updating its position, it unloads passengers whose destination matches the current stop and attempts to board new passengers if there's enough room for them and their luggage.

4.2 ALGORITHMS

4.2.1 FIFO Passenger Boarding

```
c
void loadPassengers(Bus* bus, BusStop* stop) {
    while (bus->passengerCount < 8 && stop->front <= stop->rear) {
        Passenger p = stop->queue[stop->front];
        if (bus->luggageCount + p.luggageCount <= 8) {
            bus->passengers[bus->passengerCount++] = p;
            bus->luggageCount += p.luggageCount;
            stop->front++; // Dequeue
        } else break;
    }
}
```

Description:

This function implements FIFO (First-In, First-Out) logic to load passengers from the stop queue. Only passengers with luggage that fits within the remaining luggage capacity of the bus are allowed to board. Boarding continues until the bus is full or no more valid passengers are in the queue.

4.2.2 Unloading Passengers (unloadPassengers)

```
c
void unloadPassengers(Bus* bus) {
    for (int i = 0; i < bus->passengerCount; i++) {
        if (bus->passengers[i].endStop == bus->currentStop) {
            for (int j = i; j < bus->passengerCount - 1; j++)
                bus->passengers[j] = bus->passengers[j + 1];
            bus->passengerCount--;
        } else {
            i++;
        }
    }
}
```

Description:

This function removes passengers from the bus once they reach their destination stop. If a match is found between the bus's current stop and a passenger's destination, the passenger is removed and the array is shifted to maintain order.

4.2.3 New Passenger Generation (generatePassenger)

```
c
void generatePassenger(BusStop stops[], PassengerQueue* pq) {
    Passenger p;
    p.id = globalPassengerID++;
    p.startStop = getRandomStop();
}
```

```

do {
    p.endStop = getRandomStop();
} while (p.endStop == p.startStop);

p.luggageCount = rand() % 3;
for (int i = 0; i < p.luggageCount; i++)
    p.luggageIDs[i] = globalLuggageID++;

int index = p.startStop - 'A';
if (stops[index].rear < MAX_PASSENGERS - 1) {
    stops[index].rear++;
    stops[index].queue[stops[index].rear] = p;
    if (pq->count < MAX_QUEUE)
        pq->queue[pq->count++] = p;
}
}

```

Description:

Random passengers are generated with randomized origin/destination stops and 0–2 pieces of luggage. Each is assigned a unique ID, and their luggage is uniquely tagged as well. This function adds them to the appropriate stop queue and a preview queue for display purposes.

4.3 PROBLEMS FACED

Queue Corruption: Fixed by adding bounds checks for front/rear.

UI Flickering: Resolved with system("cls") and controlled refresh rates.

Luggage Deadlocks: Implemented priority checks to prevent buses from stalling.

5. GROUP MEMBERS AND THEIR WORK

Ömer Ege Nişli	City grid initialization, core architecture
Yiğithan Yıldırım	Passenger queue logic, FIFO implementation
Asal Ghazi Mir Saeid	UI rendering, statistics display

6. CONCLUSION

This project successfully simulates a city's transport network using C structs, queues, and file I/O. Challenges in real-time rendering and queue synchronization were overcome through iterative testing. Future extensions could add graphical interfaces or multi-threading for larger-scale simulations